Semester project instructions

PHY-905-003
Computational Astrophysics and Astrostatistics
Spring 2026

1 Overall plan

The goals of the semester project in this course are: (1) for you to *broaden and/or deepen* your understanding of the numerical techniques you have learned in this class, (2) to apply this knowledge to your own research or another topic of your choice in astrophysics or physics through software *that you have written* to **analyze a dataset or model a physical situation** or **expand on select topics we worked on in class**; and (3) to share this knowledge with the rest of the class via a presentation and code demonstration. You will demonstrate achievement of these goals through several deliverables, as detailed below. Note that most deadlines are given as dates, with the time of the deadline being 11:59 p.m. that day unless otherwise specified.

In terms of project size and complexity, this is meant to be doable in the roughly six week time frame between now and finals week and take roughly as much time as two homework assignments (roughly 20 total hours of work). To that end, it may be very difficult to get something working that directly ties to your research – in that case, you should choose a project that is a simplified, constrained, and/or idealized version of what you'd actually need for your research!

Also, please note that I expect you to put all of your code, analysis scripts, data files, and so on in the directory `source_code_and_other_files` in this Git repository, and regularly commit changes to the repository and push them to GitHub (in other words, use this as the working directory for your project). This serves three key functions: (i) it allows you to keep track of your code changes, and revert those changes as needed; (ii) it provides a critical backup of your project and data in case of hardware failure, and (iii) it allows me to keep up on your progress, and follow up as necessary. So, please commit and push your changes early and often! Note also that your final code must (1) be properly formatted and commented according to the class coding standard, and (2) must be written in text files (not Jupyter notebooks!) and pass a PyLint check with no significant warnings or errors.

NOTE: I strongly encourage you to come and talk to me at all stages of this project – I'm happy to brainstorm ideas, give suggestions, and look at paper drafts and your code. Please talk to me before or after class or contact me via MatterMost to schedule an appointment at another time!

2 Project options

There are three project options. The first two options build directly on topics we've learned in class, and the third is meant to be more flexible.

2.1 Option 1 – Write a 2D hydro code

Fluid dynamics is fundamental to modeling astrophysical systems. In this context "fluid dynamics" includes both neutral fluids (as modeled with the Euler equations or Navier-Stokes equations) and plasmas (i.e., ionized and magnetized fluids, as modeled with the equations of magnetohydrodynamics). It is, as a consequence, valuable to understand how we solve these equations on a computer in order to either use these techniques yourself or to be an educated consumer of papers that use them.

In this project you will build on the work you've done in class solving hyperbolic partial differential equations – specifically, you'll extend our second-order solution of Burgers' equation (the simplest nonlinear hyperbolic PDE) to solve the Euler equations, which are the equations of inviscid hydrodynamics. You will do this by working through Chapters 7 and 8 of Michael Zingale's Computational Astrophysical Hydrodynamics lecture notes. The steps you'll take are as follows:

1. Read through Chapter 7 of Zingale's lecture notes and make sure that you understand (I) the properties of Euler's equation and (II) the Riemann problem as applied to these equations.

2. Implement a 1D finite volume hydro solver using the Piecewise Linear update (Section 8.2.2 of Zingale) and implementing the two-shock Riemann solver (as discussed in Section 8.3) and the conservative update discussed in Section 8.4. Implement a boundary condition routine that allows you to set all of the boundary conditions described in Section 8.5, as well as periodic boundary conditions. Do this assuming an ideal gas (gamma-law) equation of state.

3. Verify that your 1D hydro code works by implementing some standard test problems: specifically, the Sod shock tube and double rarefaction problem (Sec. 8.9.1), and the advection problem (Sec. 8.9.3). Show that this works with the appropriate boundary conditions, and compare the solutions (using the L2 norm) to either the initial conditions or reference solutions, as appropriate, as you vary the grid resolution and CFL condition. Verify also that your code works with both positive and negative velocities by changing the direction of the advection test and show that it recovers the same level of error. Document all of this in plots and in text!

4. After you demonstrate that your 1D code works correctly, extend it to 2D as described in Section 8.6 of Zingale. Test that this code works in each dimension individually by setting up the Sod shock tube and double rarefaction problem in the x and (separately) y directions, and then make sure that your code correctly couples the dimensions using the Sedov blast wave test (Section 8.9.2). Also use the advection test where you allow the Gaussian profile to evolve in both the $\pm x$ and $\pm y$ directions, as well as along all four diagonals, and demonstrated that it works correctly and identically in all directions.

5. Make sure to document all of your tests, including convergence and CFL tests, with plots and documentation. I strongly suggest writing the code in a way that allows you to run it via command line arguments and put out plots comparing to reference solutions!

2.2 Option 2 – Parallelize a 2D diffusion solver

NOTE: The details of this project may change depending on feedback from Claire!

The two-dimensional diffusion equation is (in Cartesian coordinates and with a constant diffusion coefficient):

$$\frac{\partial \phi}{\partial t} = k \left(\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} \right) \tag{1}$$

In this equation, ϕ is the quantity that is diffusing and k is the diffusion coefficient (note that the derivation of this is discussed in Chapter 10 of Zingale's Computational Astrophysical Hydrodynamics lecture notes) This is a convenient problem to experiment with because you can make it arbitrarily computationally expensive by changing the grid resolution and/or by changing the 3-point stencil for the second spatial derivatives to a 5-point stencil, which is a higher-order solution to the same derivative (though going to a will require you to add an additional ghost zone!).

The steps you'll take for this problem are:

1. Implement the 2D explicit diffusion method (which is a simple extension of the 1D version you wrote in class) entirely using numpy's array indexing features – in other words, don't use loops over grid indices, as this will be important later! Implement user-specifiable periodic, Dirichlet, and von Neumann boundary conditions. For initial conditions, use the Gaussian pulse problem described in Zingale (exercise 10.2). Verify that you get the correct solution as a function of time (comparing to the analytic solution using the L2 norm), and further verify that the solution in the x, y, and annularly-averaged values are all the same. Show that you get the correct convergence as you increase the grid resolution as well.

2. Parallelize your 2D problem using the Python multiprocessing module using "tiles" comprised of stripes of the array (i.e., $1/N_c$ of the domain along your y-dimension for N_c CPU cores). Using a grid whose edge (not including ghost zones) is a power of 2, measure the simulation runtime using the Python time module for the parallel component of the problem as you increase the number of CPU cores by powers of two. Do this for simulation grids of various sizes and numbers of grids, and plot the timing for both the strong and weak scaling (i.e, constant problem size vs. constant work per core), plotted vs. ideal scaling. You may wish to use a log-log plot for this! (Hint: turn off all print statements for the parallel component of the model update.) What features do you observe in this plot, and how might you interpret them? (Hint: when answering this question, consider the cost of communication and synchronization between CPU cores, and also consider Amdahl's law.) I strongly suggest running these tests on the ICER cluster, preferably using a batch job to ensure that nobody else is running on the same CPU cores as you. This will ensure that your answers are more precise!

3. Parallelize your code with either CuPy (which will work on a single GPU) or mpi4py (which uses message-based communication to parallelize across multiple CPU cores, including across multiple nodes). CuPy has an excellent tutorial that you can leverage; for mpi4py, there is a great set of mpi4py examples that you can leverage, as well as a MPI tutorial that your instructor created for the ICER+CMSE REU program, which has much more heavily annotated versions of some of those examples. For either of these examples, measure the speedup for both strong and weak scaling in the same way you did for a multiprocessing-based problem. How do these plots differ from the one you generated in the previous section? Why do you think that might be? Note that you should use the ICER cluster for these experiments, and use batch jobs to ensure that you are not sharing the GPU or CPU resources with another user (which, as discussed above, will ensure that your answers are more precise). In either case, also try using different types of hardware (V100, A100, and H200 GPUs; older-generation CPUs on the 2018 cluster vs. newer CPUs on the 2024 cluster) for one of your larger grid sizes and see if you observe any differences in behavior, and describe those!

4. Make sure to document all of your tests, including convergence and CFL tests, with plots and documentation. I strongly suggest writing the code in a way that allows you to run it via command line arguments and put out plots comparing to reference solutions!

Hint: You're going to need Python *partial functions* for the Multiprocessing component of this option. Check out this tutorial on partial functions. Here's an example of how to use a partial function:

```
my_partial = functools.partial(my_func, arg1=val1, arg2=val2)

with multiprocessing.Pool(processes=nprocs) as p:
    results = p.map(my_partial, work_to_share)
```

Alternately, you can use `mutliprocessing.Pool().starmap`, which is similar to the function taught in class (`multiprocessing.Pool().map`) but handles functions with multiple arguments. This option is useful if you don't want every process to run your desired function with the same arguments (as will happen when using a partial function).

2.3 Option 3 – Choose your own adventure

The goal of this option is for you to apply some subset of the numerical methods that you've learned in this course to an astrophysical problem (or astrophysics-adjacent problem) that's of interest to you.

One specific example of this would be to use an example problem from your own research area (or a simplified version of a problem of that type), determine what numerical methods are needed to solve that problem, solve said problem numerically, and verify that the solution behaves well numerically and makes physical sense. For example, if you were interested in supernova light curves you could write a simple implicit flux-limited diffusion or Monte Carlo radiation transport solver. If you are interested in plasma physics, you could write a 1D or 2D particle-in-cell (PIC) electrodynamic plasma code. If you are interested in understanding the light curve of an eclipsing multi-planet solar system you could write a Bayesian fitting tool that provides solutions (and estimates for accuracy) for fits to a light curve.

Alternately, you could find a peer-reviewed journal article that uses some combination of numerical methods to solve a problem that you are interested in. Implement your own version of the algorithm(s) that they used to solve that problem and verify that you get the same answers. If you end up getting *different* answers, try to understand why and explain it. (And appreciate that not all peer-reviewed journal articles are correct, since the peer review system is imperfect!)

For your project proposal, briefly explain (i) the scientific motivation for the project, (ii) how it relates to your research and/or your scientific interests, (iii) the numerical methods that you will learn about and implement for this project and how this goes beyond what we've learned in class in either breadth or depth, and (iv) your intended deliverables in terms of data analysis products or model outputs.

3 Project components and deadlines

1. **Project proposal, due Wednesday 3/18/2026**. Write a brief ($\simeq 1-2$ paragraph) proposal describing which project option you plan to pursue. For Options 1 and 2 just explain why you're interested in this project. For Option 3 ("choose your own adventure") please go into detail as explained in Section 2.3. Turn this propsal in via a text or markdown file in the `proposal` subdirectory in this repository. In class the next day we'll have a brief roundtable for everybody to explain their proposed project, and I will also follow up with written feedback (and may ask you to modify your proposal if you pursue Option 3 and it's overly ambitious or needs further clarification).

2. **Project update #1, due Monday 3/30/2026**. The goal for this update is for you to have done the necessary background reading and started implementation of the code you need for your project, but not necessarily have a fully-working code (though what you do have should be in `source_code_and_other_files` and committed to the repository). The written update go in the subdirectory `update_1`. For Options 1 and 2, please focus on the challenges that you've experienced so far and the strategies you plan to use to overcome those challenges. For Option 3 this document should include an update on your proposed project that details (i) the numerical methods you're using and (if relevant) the sources you've used to learn about them, (ii) your progress thus far, and (iii) a discussion of any unexpected challenges you've run into and the steps you've taken to deal with these challenges. Note: if your project is going poorly this is a good time to re-evaluate your project and revise your plans, which may include changing its scope! We will also have a brief roundtable in class the following day for everybody to give a quick project update, highlighting status and challenges.

3. **Project update #2, due Wednesday 4/15/2026**. At this point, your code should be close to complete in terms of implementation of its core functionality, and you should be able to verify that it behaves as expected using simple datasets (if you've created an analysis tool) or a simple simulation setup (if you've written a model of some sort). Note that these simple setups may be useful as tests! As with the previous update, your code should be in the `source_code_and_other_files` directory and your written update goes in the subdirectory `update_2`. For all options the written update should describe (i) your progress thus far and (ii) what remains to be done before your final submission. For Option 3, please also inclue what the tool/model is intended to do and why you believe that it is behaving as expected based on your simpl dataset/model setup. **By this point, your code should also adhere to the class coding standard and pass checks with PyLint.** We'll have a brief set of updates in class the day following this deadline.

4. **Project presentations and code demonstrations, due before the course's final exam session the week of finals (the week of 4/27/2026 – exact date/time for this is TBD)**. Your final presentation can use any of the standard presentation file formats (Powerpoint, Keynote, Google Slides, etc.), and an exported PDF version should be put in the `presentation_demo` subdirectory. Presentations will be a total of 10 minutes long, including 7-8 minutes for your talk and 2 minutes for questions and discussion. Make sure that your presentation includes the justification for your project and your goals (i.e., what your code is supposed to accomplish), the numerical methods used (including a brief explanation of how the new methods you have learned about work!), the results that you obtained, and any lessons that you learned about the methods or implementation that you think would be interesting to your colleagues. If the project is an idealized, reduced, etc. version of something you'd do for your

research, please also explain that! You should also be prepared to give a brief demonstration of your code, if necessary. Your presentation will be evaluated on the clarity of your visual aids as well as the quality, clarity, and delivery of your presentation. Please note that talk slides are due via GitHub immediately before we start presentations.

5. **Project final code deadline, due Friday 5/1/2026**. This is the final deadline for your code, all of which should go in the `final_code` subdirectory. **Note that this is a firm deadline – grades are due a few days after this, and I need time to go through everything!**

   Your code should: (1) be properly formatted and commented according to the class coding standard, and (2) must pass a PyLint check with no *significant* warnings or errors. Comments at the top of the main source file must describe how to run the code, including a listing of any additional Python packages that must be installed (on top of a baseline Miniconda distribution running a recent version of Python 3) in order to get your code to work. Your code will be assessed on your implementation of the chosen algorithms, the results that it produces, code clarity/modularity/comments, and whether it runs on my Mac laptop and/or on ICER's clusters when using the aforementioned Python distribution. In addition to your code, please include any data files that are required, as well as scripts used to generate plots or do additional analysis. (If the datasets are larger than $\simeq$ 10 megabytes, please talk to me before committing them to the repository - there are other, better solutions than Git for large files!)